MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasile Brînză, FAF-242

Report

Laboratory Work No. 3

Cryptography and Security

Topic: Polyalphabetic Ciphers (Vigenere Cipher)

Checked by:
Zaica Maia, *university assistant*

Chişinău 2026

Table of Contents

1 INTRODUCTION....3
1.1 Polyalphabetic ciphers and why they exist....3
1.2 Purpose of the laboratory work....3
2 THEORETICAL BACKGROUND....3
2.1 How the Vigenère cipher works....3
2.2 Why it is stronger than Caesar....3
2.3 Using the Romanian alphabet (31 letters)....4
3 IMPLEMENTATION....5
3.1 The alphabet....5
3.2 Checking the input....5
3.3 The Vigenère function....5
3.4 The main function....6
4 TESTING....6
4.1 Encryption and decryption....6
4.2 Wrong input....7
CONCLUSION....8
REFERENCES....9

1 INTRODUCTION

1.1 Polyalphabetic ciphers and why they exist

A monoalphabetic cipher, like Caesar, always replaces a letter with the same letter, so the letter frequencies of the language stay visible in the ciphertext. This is the weak point we used in laboratory work number 2 to break a cryptogram. A polyalphabetic cipher fixes this by using several substitution alphabets: the same plaintext letter can become different ciphertext letters, depending on its position in the text. This flattens the frequency distribution, so a simple frequency count no longer gives the answer. The best-known example is the Vigenère cipher.

1.2 Purpose of the laboratory work

The purpose of this laboratory work is to understand how the Vigenère cipher works and to implement it in a program.

Task. My variant is 4 (even number), so I have Task 3.2: implement the Vigenère cipher for Romanian messages. The alphabet has 31 letters, coded with the numbers 0, 1, ..., 30. The program must:

- accept only the letters A–Z and a–z, and show the correct range if the user types anything else;
- require a key of at least 7 letters;
- remove the spaces from the message and turn all letters into capitals;
- let the user choose encryption or decryption, enter the key and the message (or the ciphertext), and show the result;
- use the formulas from the mathematical model of the cipher.

The program is written in Python.

2 THEORETICAL BACKGROUND

2.1 How the Vigenère cipher works

The Vigenère cipher uses the same operation as Caesar (shifting a letter), but the key is a word, not a single number. Every letter of the key gives one shift. The key is written again and again under the message, and each message letter is shifted by the key letter under it. Encryption and decryption use these formulas [2]:

$$c_i = (m_i + k_i) \bmod 31 \quad (1)$$

$$m_i = (c_i - k_i) \bmod 31 \quad (2)$$

Here m_i is the code of the plaintext letter, k_i is the code of the key letter, and c_i is the code of the ciphertext letter. For decryption the key letter is subtracted instead of added.

2.2 Why it is stronger than Caesar

The Caesar cipher has only one shift, so there are just a few keys to try. In Vigenère the shift changes from letter to letter, so the same plaintext letter can be encrypted in different ways, and the frequencies of the

language are hidden. An attacker also does not know the length of the key. For example, a key of 5 letters on a 26-letter alphabet already gives $26^5 = 11\ 881\ 376$ possible keys [2].

2.3 Using the Romanian alphabet (31 letters)

The lab asks for the Romanian alphabet, so the calculation is done modulo 31, not 26. The letters get the codes 0–30 in this order:

A=0, Ă=1, Â=2, B=3, C=4, D=5, E=6, F=7, G=8, H=9, I=10, Î=11, J=12, K=13, L=14, M=15, N=16, O=17, P=18, Q=19, R=20, S=21, Ș=22, T=23, Ț=24, U=25, V=26, W=27, X=28, Y=29, Z=30.

Example with the key SUPERBAN. The first letter P (18) is shifted by S (21): (18 + 21) mod 31 = 8, which is G. The second letter E (6) is shifted by U (25): (6 + 25) mod 31 = 0, which is A. The sum went over 30 and "wrapped around", this is what mod 31 does. To decrypt, A (0) minus U (25) gives −25, and (−25) mod 31 = 6, which is E again.

3 IMPLEMENTATION

The full code is in my repository [1], in the folder of this lab. Below are its four parts.

3.1 The alphabet

```
lab3 > main.py > ...
1    ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
2    LATIN = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
3    N = 31
4
```

`ALPHABET` has the 31 Romanian letters. The position of a letter in this string is its code (A is 0, Ă is 1, ..., Z is 30), so no separate table is needed. `LATIN` has the only letters the user is allowed to type (A–Z). `N` is the modulus.

3.2 Checking the input

```
5   def text_formatting(text, alphabet):
6       text = text.upper() # conversion to uppercase
7       text = text.replace(" ", "") # remove spaces
8       text = text.replace("Ş", "Ș").replace("Ţ", "Ț") # adjust for cedilla edge case
9
10      if text == "":
11          print("Input text is empty")
12          return None
13
14      for char in text:
15          if char not in alphabet:
16              print(f"Invalid character '{char}' in the text. Please use only characters from the alphabet '{alphabet}'.")
17              return None
18
```

This function prepares the text as the lab requires: it makes all letters capital and removes the spaces. Then it checks every character. If one is not in the allowed alphabet, it prints the correct range and returns `None`, so the program asks for the input again. The same function is used for the key, the message and the ciphertext. For the ciphertext the allowed alphabet is the full one, because encryption can give Ă, Â, Î, Ș or Ț.

3.3 The Vigenère function

```
21  def vigenere(text, key, sign):
22      result = ""
23      for i in range(len(text)):
24          m = ALPHABET.index(text[i])            # letter code from the text
25          k = ALPHABET.index(key[i % len(key)])  # letter code from the key
26          result += ALPHABET[(m + sign * k) % N]
27      return result
28
```

One function does both operations: `sign` is 1 for encryption and -1 for decryption. For every letter it takes the code of the text letter (`m`) and the code of the key letter (`k`). The expression `i % len(key)` repeats the key when the text is longer. The line `(m + sign * k) % N` is the formula from Section 2.1. In Python, `%` gives a positive result even for negative numbers, so decryption works without extra code.

3.4 The main function

```
def main():
    # encrypt / decrypt choice
    while True:
        op = input("Choose an operation (e / d): ")

        if op in ["e", "d"]:
            break
        else:
            print("Invalid operation. Please choose 'e' for encryption or 'd' for decryption.")

    # keyword
    while True:
        key = text_formatting(input("Enter a keyword (at least 7 letters): "), LATIN)

        if key is None:
            continue

        if len(key) < 7:
            print("Invalid keyword. Please enter a keyword with at least 7 characters")
            continue

        break

    # text for encryption / decryption
    while True:
        if op == "e":
            text = text_formatting(input("Enter a message: "), LATIN)
        else:
            text = text_formatting(input("Enter a ciphertext: "), ALPHABET)

        if text is not None:
            break

    if op == "e":
        print("Ciphertext:", vigenere(text, key, 1))
    else:
        print("Message:", vigenere(text, key, -1))

if __name__ == "__main__":
    main()

```

The program asks for three things, each in its own loop that repeats until the value is valid: the operation, the key (at least 7 letters) and the text. At the end it calls `vigenere` with 1 or -1 and prints the result.

4 TESTING

4.1 Encryption and decryption

I tested the program with two keys. Every ciphertext was decrypted back to the original message, so the two operations are correct. The third test shows that the result can contain Romanian letters (Ș) even if the message has only A–Z.

Table 4.1: Tests of encryption and decryption

The program output for test 1 is shown in Figure 4.1.

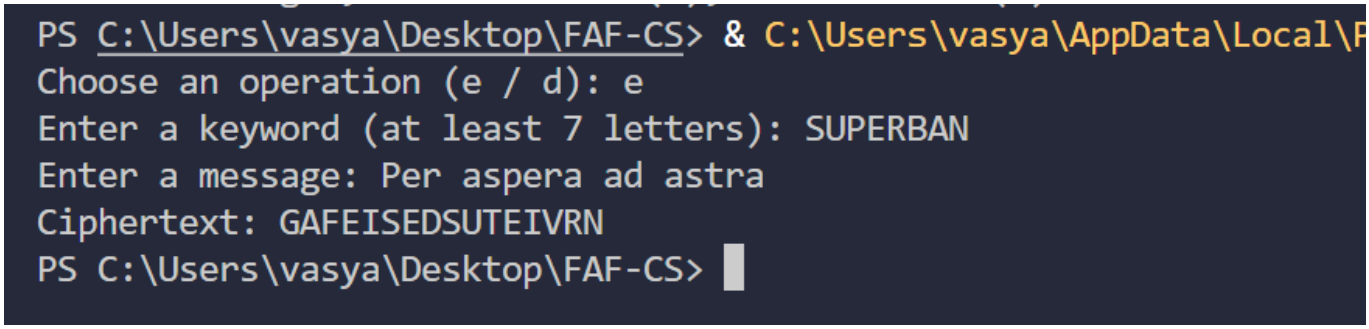


Figure 4.1 – Program output for encryption

The result is different from the 26-letter example in the lab guide (HYGEJHYGERVUHXIS for the key SUPER), because here the alphabet has 31 letters and the key is different.

4.2 Wrong input

The program does not stop when the input is wrong, it explains the problem and asks again (Figure 4.2).

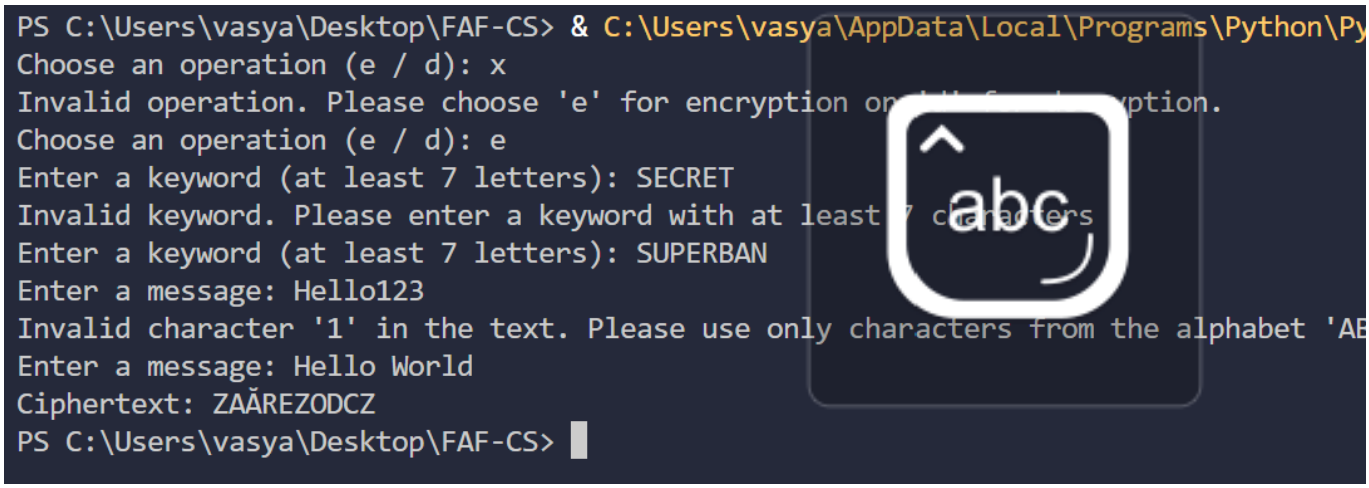


Figure 4.2 – Program output for wrong input

The wrong operation, the key with 6 letters and the message with digits were all refused. In the last line the space was removed from “Hello world” and the letters were made capital before encryption, as the task requires.

CONCLUSION

In this laboratory work I implemented the Vigenère cipher in Python for the Romanian alphabet of 31 letters (Task 3.2, variant 4). The program removes spaces, makes the text capital, accepts only A–Z, asks for a key of at least 7 letters, and encrypts or decrypts with the formulas c = (m + k) mod 31 and m = (c − k) mod 31. The tests showed that decryption always returns the original message and that wrong input is refused with a clear message.

The main difference from the monoalphabetic ciphers of the previous labs is that the shift changes with the key letter, so the same letter is not always encrypted in the same way. The most useful detail in the code was %: it keeps every code between 0 and 30, also for decryption, where the result of the subtraction can be negative.

Vigenère is stronger than Caesar, but it is still not secure by today's standards: the key repeats, so a long text can be attacked once the key length is found, and a short key makes it easier. Modern ciphers do not have this weakness.

REFERENCES

[1] GitHub repository. [Online]. Available: https://github.com/kynexi/FAF-CS. [Accessed: Oct. 29, 2026].